TinyIDS deployment: Pi + NodeMCU + laptop

Run the full pipeline on real hardware, step by step

07 November 2026. Folder: `project_final/deploy/`. Pi IP in this guide: `172.29.1.135` (check yours with `hostname -I`).

Contents

1 What runs where **2**

2 The folder **2**

3 Step A: laptop libraries (once) **3**

4 Step B: copy the Pi part to the Pi **3**

5 Step C: Pi one-time setup **3**

6 Step D: Pi: start our broker and the detector **4**

7 Step E: laptop: dashboard **4**

8 Step F: NodeMCU: the bot sketch **4**

9 Step G: laptop: the attacks **5**

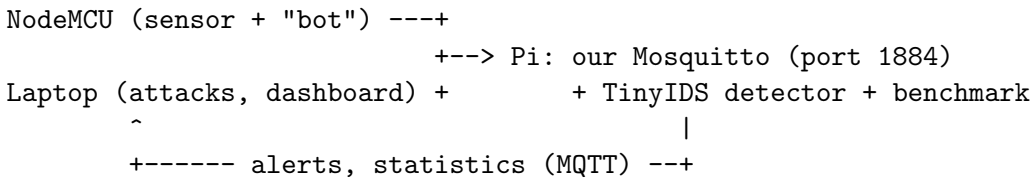
10 Step H: laptop: the NodeMCU as a bot **5**

11 Step I: Pi: the benchmark (RQ2) **5**

12 Step J: clean up on the Pi **6**

13 Troubleshooting **6**

1 What runs where



- **Pi = gateway**: our own MQTT broker (Mosquitto, port **1884**, with a password) and the **TinyIDS detector** (packet capture → features → models → SHAP → alerts). The lab's own broker on port 1883 is not touched.
- **NodeMCU**: a normal sensor that can be turned into a "bot" by a command from the laptop.
- **Laptop**: the attacks and the live dashboard.

Abbreviations: MQTT = Message Queuing Telemetry Transport; IDS = Intrusion Detection System; SHAP = SHapley Additive exPlanations; SSH = Secure Shell; venv = Python virtual environment; RAM = Random Access Memory.

2 The folder

File	What it is
`pi/run_tinyids.py`	Main program: connects all the parts below (class `TinyIDS`).
`pi/tinyids/settings.py`	All settings in one place (interface, port, window sizes, limits).
`pi/tinyids/features.py`	`PacketReader`, `FlowTable`, `FeatureBuilder`: packets → the 30 dataset features.
`pi/tinyids/windows.py`	`WindowCollector`: per sender, windows of 10 and 100 packets.
`pi/tinyids/detector.py`	`Detector`: binary model, family model, SHAP top-3.
`pi/tinyids/rules.py`	`BruteForceRule`: many refused MQTT logins = brute force.
`pi/tinyids/publisher.py`	`AlertPublisher`: sends alerts / statistics over MQTT.
`pi/tinyids/stats.py`	`Stats`: counters for the dashboard.
`pi/models/`	The two LightGBM models + `meta.json` (feature list, classes).
`pi/broker/tinyids_broker.conf`	Config of our broker (port 1884, password required).
`pi/setup_pi.sh, pi/run_ids.sh`	The setup commands and the start commands, for sharing.
`pi/benchmark/rq2_benchmark.py`	RQ2 benchmark: size, RAM, speed, accuracy, energy (+ test data and 7 models).
`laptop/dashboard.py`	Live dashboard (Streamlit).
`laptop/attacks/*.py`	`attack_base.py` (shared sending loop) + SYN flood, UDP flood, port scan, MQTT brute force.
`laptop/bot_master.py`	Sends commands to the NodeMCU bots.
`nodemcu/nodemcu_bot/nodemcu_bot.ino`	NodeMCU sketch: sensor + bot.

3 Step A: laptop libraries (once)

```
cd C:\Users\HOME\Documents\iot\project_final\deploy\laptop
pip install -r requirements.txt
```

Installs `scapy` (builds attack packets), `paho-mqtt`, `streamlit` + `pandas` (dashboard).

4 Step B: copy the Pi part to the Pi

In laptop PowerShell:

```
scp -r C:\Users\HOME\Documents\iot\project_final\deploy\pi pi@172.29.1.135:~/tinyids
```

`scp` = copy over SSH. The `pi` folder becomes `~/tinyids` on the Pi.

5 Step C: Pi one-time setup

In the SSH window, type one line at a time:

```
cd ~/tinyids
PROXY=http://172.31.2.3:8080
sudo date -s "$(curl -sI -x $PROXY https://www.google.com | grep -i '^date:' | cut -d' ' -f2-)"
python3 -m venv venv
./venv/bin/pip install --proxy $PROXY numpy lightgbm scapy paho-mqtt onnxruntime psutil
dpkg -s libgomp1 libpcap0.8 >/dev/null 2>&1 || sudo apt install -y libgomp1 libpcap0.8
mosquitto_passwd -c -b broker/passwd tinyids tinyids123
chmod 0700 broker/passwd
./venv/bin/python -c "import lightgbm, scapy, onnxruntime, paho.mqtt; print('all OK')"
```

1. **Clock**: the college blocks the time server, so the time is read from Google through the proxy.
2. **venv**: a private Python only for this project (Debian blocks `pip` in the system Python).
3. **Packages** through the college proxy (about 80 MB).
4. **System libraries** for LightGBM and packet capture, installed only if missing.
5. **Password** for our broker (user `tinyids`); `chmod 0700` = only you can read it (Mosquitto requires this).
6. **Check**: must print `all OK`.

Note: no "(venv)" in the prompt is normal. It only appears after `source venv/bin/activate`. We call `./venv/bin/pip` and `./venv/bin/python` directly instead, which does the same, and also works with `sudo` (which ignores an activated venv).

Undo everything: `rm -rf ~/tinyids`.

6 Step D: Pi: start our broker and the detector

```
cd ~/tinyids
mosquitto -c /home/pi/tinyids/broker/tinyids_broker.conf -d
ss -ltn | grep 1884
sudo ./venv/bin/python run_tinyids.py --interface wlan0
```

- `mosquitto ... -d`: starts *our* broker in the background (`-d` = daemon).
- `ss -ltn | grep 1884`: checks it is listening; must show `LISTEN ... :1884`.
- `sudo ./venv/bin/python run_tinyids.py`: the detector (`sudo` because capturing packets needs root). It prints `TinyIDS watching wlan0 (172.29.1.135), broker port 1884`. **Keep this window open**: alerts are printed here.

(`./run_ids.sh` does both start commands in one go.)

7 Step E: laptop: dashboard

New PowerShell window:

```
cd C:\Users\HOME\Documents\iot\project_final\deploy\laptop
streamlit run dashboard.py -- 172.29.1.135
```

Open **http://localhost:8501**. Top left must say **Broker: connected**. The `--` separates Streamlit's own options from the dashboard's options (the Pi's IP).

Dashboard on the Pi's own monitor? Possible: `./venv/bin/pip install --proxy $PROXY streamlit pandas`, then `./venv/bin/streamlit run dashboard.py -- 127.0.0.1` on the Pi (copy `dashboard.py` there first). Not recommended during measurements: Streamlit and a browser use a lot of the Pi's CPU and RAM and distort the benchmark numbers.

8 Step F: NodeMCU: the bot sketch

1. Arduino IDE: open `deploy/nodemcu/nodemcu_bot/nodemcu_bot.ino`.
2. Check the settings at the top: `WIFI_SSID "IIITA"`, `WIFI_PASS ""`, `GATEWAY_IP "172.29.1.135"`, `MQTT_PORT 1884`.
3. Board **NodeMCU 1.0 (ESP-12E Module)**, the right COM port, **Upload**.
4. Serial Monitor at **9600**: `Wi-Fi OK`, then `MQTT 172.29.1.135:1884 ... OK`, then a reading every 2 s.
5. The dashboard's Sensors table shows **node1**.

Known issue: the model may flag the NodeMCU's normal traffic as "Spoofing" (its normal traffic looks different from the dataset's). Ignore for now.

9 Step G: laptop: the attacks

PowerShell **as Administrator** (scapy needs it to send raw packets on Windows):

```
cd C:\Users\HOME\Documents\iot\project_final\deploy\laptop\attacks
python syn_flood.py 172.29.1.135 --seconds 10
python udp_flood.py 172.29.1.135 --seconds 10
python port_scan.py 172.29.1.135 --last 200
python mqtt_bruteforce.py 172.29.1.135
```

Attack	What it sends	Expected alert
SYN flood	1,000 TCP "let's connect" packets/s to port 1884, never finished	Flood
UDP flood	1,000 small junk UDP packets/s to port 5683	Flood
Port scan	one SYN to each port 1–200; prints the open ports	Recon (or Flood: it is fast)
MQTT brute force	tries common passwords for user `tinyids` on port 1884	BruteForce (rule)

Wait about 10 s between attacks; watch the Pi window and the dashboard. Defaults: 10 s, 1,000 packets/s, only the Pi's IP.

10 Step H: laptop: the NodeMCU as a bot

```
cd C:\Users\HOME\Documents\iot\project_final\deploy\laptop
python bot_master.py 172.29.1.135 udp_flood --seconds 10
python bot_master.py 172.29.1.135 tcp_flood --seconds 10
python bot_master.py 172.29.1.135 stop
```

The laptop publishes a command on topic `cmd/bot`; the NodeMCU floods the Pi (the only target, fixed in the sketch) and returns to normal after the time (max 30 s). The alert's source IP is the NodeMCU's IP. Bot messages appear on `bot/node1/status`.

11 Step I: Pi: the benchmark (RQ2)

First stop the detector (**Ctrl+C**) so it does not disturb the measurement.

```
cd ~/tinyids
./venv/bin/python benchmark/rq2_benchmark.py
./venv/bin/python benchmark/rq2_benchmark.py --energy mlp_int8
```

- The first command measures 7 models (IDS LightGBM, pruned LightGBM / Random Forest in ONNX, MLP and 1D-CNN in fp32 and INT8), each in a fresh process: **size (KB)**, **RAM (MB)**, **time per window (ms)**, **throughput (windows/s)**, **SHAP time (ms)**, accuracy and macro-F1 on 20,000 test windows. Saved to `benchmark/results_pi.csv`.
- The second is the **energy** test and needs the **USB power meter**: a small **hardware** device plugged between the Pi's power adapter and the Pi, with a screen showing watts. The script has an idle phase and a busy phase of 30 s each; you read the meter in both and type the two

numbers; it computes **mJ per 1,000 windows**. The Pi 4 has no built-in power sensor, so there is no software replacement.

12 Step J: clean up on the Pi

```
pkill -f tinyids_broker.conf
```

Stops *our* broker only. The lab's broker on port 1883 keeps running.

13 Troubleshooting

Problem	**Fix**
`scp`: connection refused / wrong IP	Check the Pi's IP with `hostname -I`.
`pip`: connection errors on the Pi	Proxy missing: add `--proxy http://172.31.2.3:8080`.
`apt`: 403 Forbidden / "Not live until"	Proxy file for apt missing / Pi clock wrong (see Pi gateway guide).
Mosquitto does not start (Step D)	Password file missing or wrong rights: redo the two `passwd` lines of Step C.
Dashboard: "Broker: offline"	Our broker not running (Step D), wrong IP, or laptop on another network.
Attack scripts: permission / Npcap errors	Run PowerShell as Administrator.
NodeMCU: `failed (state 5)`	Wrong MQTT user/password in the sketch.
NodeMCU: `failed (state -2)`	Wrong gateway IP or port, or our broker not running.